

Job Application Assistant - Toteutussuunnitelma

Kattava toteutussuunnitelma paikallisesti ajettavalle Python-tyokalulle.

Sisältö pohjautuu projektille määriteltyyn MVP-ajatteluun, laajennettavaan arkkitehtuuriin, testaukseen ja tietosuojaperiaatteisiin.

Versio 0.1

Job Application Assistant - Toteutussuunnitelma

1. Tavoite

Projektin tarkoitus on rakentaa paikallisesti ajettava Python-työkalu, joka analysoi työpaikkailmoituksen sisällön suhteessa hakijan CV:hen ja auttaa muodostamaan kohdennetumman hakemuksen. Työkalu ei laheta dataa ulkoisiin palveluihin, vaan koko analyysi voidaan suorittaa paikallisesti yksityisyyden suojaamiseksi.

Työkalu tuottaa käyttäjälle neljä konkreettista lopputulosta:

- lista työpaikkailmoituksesta tunnistetuista avainsanoista ja osaamisista
- vertailu siihen, miten hyvin samat teemat näkyvät CV:ssä
- suositukset siitä, mitä hakemuksessa kannattaa erityisesti korostaa
- luonnos osiosta "vahvuudet tähän rooliin"

2. Käyttäjatarve ja arvo

Moni työhakemus kaatuu siihen, että hakija osaa paljon oikeita asioita, mutta ne eivät näy tarpeeksi selkeästi juuri kyseisen roolin kielellä. Työkalu ratkaisee tämän ongelman kolmella tavalla:

- se auttaa tunnistamaan työpaikkailmoituksen ydintermit nopeasti
- se osoittaa, missä kohtaa CV tukee roolia hyvin ja missä on katveita
- se auttaa muotoilemaan hakemuksen painotukset vakuuttavasti mutta konkreettisesti

Käyttäjän saama arvo on ajansäästö, parempi osuvuus hakemuksiin ja yhtenäisempi tapa rakentaa roolikohtaisia hakemuksia.

3. MVP:n rajaus

Ensimmäinen julkaisu kannattaa rajata selkeäksi komentorivityökaluksi. MVP:ssä keskitytään siihen, että ydinanalyysi toimii hyvin ilman raskaita riippuvuuksia.

MVP sisältää:

- tekstimuotoisen työpaikkailmoituksen lukeminen tiedostosta tai suoraan syötteenä
- tekstimuotoisen CV:n lukeminen tiedostosta tai suoraan syötteenä
- avainsanojen poiminta työpaikkailmoituksesta
- osaamisten ja teemojen osumien tunnistus CV:stä
- painotussuosittelujen generointi
- "vahvuudet tähän rooliin" -luonnoksen tuottaminen
- tulosten tulostus konsoliin ja tallennus Markdown-raportiksi

MVP ei vielä sisällä:

- selainkäyttöliittymää
- pilvipalveluriippuvuuksia
- automaattista hakemuksen lähetystä
- monimutkaista koneoppimismallia
- PDF- tai DOCX-sisäänlukua ensimmäisessä vaiheessa, ellei se oteta erillisenä laajennuksena

4. Toiminnalliset vaatimukset

Työkalun tulee pystyä:

- lukemaan tyopaikkailmoituksen teksti UTF-8 TXT- tai Markdown-tiedostosta
- lukemaan CV:n teksti UTF-8 TXT- tai Markdown-tiedostosta
- puhdistamaan ja normalisoimaan tekstit analyysia varten
- tunnistamaan roolin kannalta keskeisiä avainsanoja, teknologioita, pehmeitä taitoja ja vastuuteemoja
- laskemaan, mitkä avainsanat esiintyvät CV:ssa suoraan tai lähes vastaavina ilmaisuina
- erottelamaan vahvat osumat, osittaiset osumat ja puuttuvat teemat
- ehdottamaan 5-10 painotusta hakemustekstiin
- muodostamaan luonnoksen roolikohtaisesta vahvuusosiosta
- tallentamaan analyysin tiedostoksi jatkokayttöä varten

5. Ei-toiminnalliset vaatimukset

Ratkaisun tulee olla:

- paikallisesti ajettava, jotta arkaluonteinen CV-data pysyy omalla koneella
- selitettävissä, eli jokaisen ehdotuksen yhteydessä tulee olla perustelu tai viite löydettyyn tekstiin
- nopea, tavoite alle 3 sekuntia tavallisille tekstimassoille
- laajennettava, jotta uusia formaatteja ja analyysikerroksia voidaan lisätä myöhemmin
- testattava, jotta muutokset eivät riko analyysin peruslogiikkaa
- kielijoustava, jotta suomi- ja englanninkieliset ilmoitukset toimivat kohtuullisen hyvin

6. Käyttötapaukset

Keskeiset käyttötapaukset ovat:

- hakija syöttää yhden tyopaikkailmoituksen ja yhden CV:n ja pyytää analyysia
- hakija haluaa nopeasti listan ilmoituksen ydinsanoista
- hakija haluaa nähdä, mitkä asiat CV:ssa jo tukevat roolia
- hakija haluaa tiedon siitä, mitä kannattaa nostaa esiin motivaatiokirjeessä
- hakija haluaa valmiin ensiluonnoksen vahvuusosiosta, jota voi itse muokata

7. Ehdotettu arkkitehtuuri

Ratkaisu kannattaa jakaa kuuteen paakomponenttiin:

- ingestion: tiedostojen luku ja tekstin sisäänotto
- preprocessing: normalisointi, tokenisointi, stop-sanojen suodatus ja fraasien muodostus
- extraction: avainsanojen ja vaatimusteemojen poiminta tyopaikkailmoituksesta
- matching: vertailu CV:n sisältöön ja osumien pisteytys
- recommendation: painotusten ja vahvuusluonnoksen generointi
- reporting: tulosten esitys konsolissa ja tiedostona

Tämä rakenne mahdollistaa sen, että analyysilogiikka, tekstinkäsittely ja raportointi pysyvät erillisinä ja helposti testattavina.

8. Tekstinkasittely ja analyysilogiikka

Perusputki toimii seuraavasti:

- 1 Luetaan tyopaikkailmoituksen teksti ja CV.
- 2 Muunnetaan teksti pienaakkosiksi ja poistetaan ylimääräinen valimerkki- ja whitespace-kohina.
- 3 Jaetaan teksti sanoihin ja 2-3 sanan fraaseihin.
- 4 Suodatetaan yleisimmat stop-sanat suomen ja englannin listoilla.
- 5 Tunnistetaan mahdolliset osaamistermit, teknologiat, vastuut ja pehmeät taidot.
- 6 Verrataan ilmoituksen termejä CV:n termeihin taysina ja lahein osumina.
- 7 Pisteytetään teemat osuman vahvuuden, toistuvuuden ja kontekstin perusteella.
- 8 Muodostetaan suositukset ja luonnosteksti.

Avainsanojen poiminnassa kannattaa yhdistää useita heuristiikkoja:

- taajuus suhteessa tekstin pituuteen
- esiintyminen otsikoissa ja luettelokohdissa
- tunnistetut teknologiat termit ja roolisanasto
- kaksisanaiset ja kolmisanaiset fraasit, kuten "project management" tai "asiakasrajapinta"
- ilmaukset, jotka esiintyvät vaatimusten yhteydessä, kuten "edellytämme", "toivomme", "required", "preferred"

9. Pisteytysmalli

Jokaiselle tunnistetulle teemalle voidaan laskea yhdistetty pistemaara esimerkiksi seuraavista osista:

- ilmoituksen relevanssipaino 0-5
- suora osuma CV:ssä 0-5
- osittainen tai synonyymiosuma 0-3
- kontekstipaino 0-2 sen mukaan, esiintyykö asia kokemuksen, saavutuksen tai vastuun yhteydessä

Tuloksena jokaiselle teemalle saadaan status:

- vahva osuma
- kohtalainen osuma
- puuttuva tai heikko osuma

Taman perusteella voidaan muodostaa myös kokonaiskuva siitä, kuinka hyvin CV tukee kyseistä roolia.

10. Suositusten muodostus

Suosituslogiikan tavoite ei ole keksia taitoja, joita CV ei sisalla. Sen sijaan sen tulee nostaa esiin asiat, joille löytyy tukea CV:sta mutta joita kannattaa korostaa paremmin hakemuksessa.

Suosituksia voidaan jakaa kolmeen ryhmään:

- korosta tata heti alussa: asiat, joissa CV tukee vaatimusta vahvasti
- konkretisoi esimerkillä: asiat, joissa osaaminen on olemassa mutta näyttö kannattaa sanoittaa paremmin
- huomioi varovasti: asiat, joihin on osittainen osuma tai joista kannattaa kirjoittaa oppimishalukkuuden kautta

Jokaisen suosituksen yhteyteen tulee liittää lyhyt perustelu, esimerkiksi mihin CV:n kohtaan ehdotus nojaa.

11. Vahvuudet tahan rooliin -luonnos

Luonnoksen generointi kannattaa rakentaa mallipohjaisesti. Teksti muodostuu seuraavista elementeistä:

- avaava virke, joka peilaa roolin ydintarvetta
- 2-4 vahvuutta, jotka linkittyvät suoraan ilmoituksen avaintemoihin
- jokaiselle vahvuudelle lyhyt konkreettinen ankkuri CV:sta
- lopetus, joka sitoo osaamisen roolin tavoitteisiin

Esimerkkirakenne:

- Olen erityisen vahva roolissa, jossa korostuvat X, Y ja Z.
- Kokemukseni A:sta tukee tehtävän vastuualuetta B.
- Lisäksi taustani C:ssa auttaa tuomaan arvoa tilanteissa, joissa tarvitaan D:tä.
- Uskon, että voisin tuoda rooliin sekä nopeaa kynnistymiskykyä että kehittävää otetta.

12. Ehdotettu projektirakenne

Projektin kansiorakenne kannattaa rakentaa seuraavasti:

- src/job_application_assistant/: varsinainen sovelluskoodi
- src/job_application_assistant/cli.py: komentoriviliittymä
- src/job_application_assistant/io.py: tiedostonluku ja syötteen hallinta
- src/job_application_assistant/preprocess.py: normalisointi ja tokenisointi
- src/job_application_assistant/extract.py: avainsanojen poiminta
- src/job_application_assistant/match.py: osumien laskenta ja pisteytys
- src/job_application_assistant/recommend.py: suositukset ja luonnostekstin generointi
- src/job_application_assistant/report.py: raportin muodostus
- tests/: yksikkö- ja integraatiotestit
- docs/: suunnitelmat, esimerkit ja tuotetut raportit

13. Komentoriviliittymän ehdotus

Ensiversioon sopiva käyttötapo:

- ``python -m job_application_assistant analyze --job job.txt --cv cv.txt``
- ``python -m job_application_assistant analyze --job job.md --cv cv.md --output report.md``
- ``python -m job_application_assistant keywords --job job.txt``

Komentorivin tulisi tukea ainakin seuraavia valintoja:

- syötetiedostojen polut
- ulostuloraportin polku
- kieliasetus auto tai manual
- maksimiavainsanojen määrä
- tulostusmuoto console tai markdown

14. Teknologiaapaatokset

Suosittelutoteutusmalli vaiheeseen 1:

- Python 3.11+
- standardikirjasto ensisijaisena
- ``argparse`` tai ``typer`` komentoriville
- ``dataclasses`` tai ``pydantic`` rakenteisiin
- ``pathlib`` tiedostopolkujen hallintaan
- ``re``, ``collections`` ja ``difflib`` perusanalyysiin

Vaiheen 2 mahdolliset lisät:

- ``rapidfuzz`` parempaan epätarkan osuman tunnistukseen
- ``spacy`` tai muu NLP-kirjasto, jos halutaan kehittyneempää sanaluokka- tai entiteettitunnistusta
- ``rich`` raportoinnin parantamiseen konsolissa
- ``jinja2`` mallipohjien hallintaan

15. Toteutusvaiheet

Suositteluteteneminen seitsemässä vaiheessa:

- 1 Projektirungon perustaminen ja CLI-sisäänmeno.
- 2 Tekstin luku, normalisointi ja stop-sanojen hallinta.
- 3 Avainsanojen ja fraasien poiminta tyopaikkailmoituksesta.
- 4 CV-vertailu, osumaluokittelu ja pisteytys.
- 5 Suosituslogiikka ja vahvuusluonnoksen generointi.
- 6 Markdown-raportin muodostus ja käytettävyyden viimeistely.
- 7 Testit, esimerkkisyötteet ja dokumentaatio.

16. Testausstrategia

Testauksen tulee kattaa ainakin:

- tiedostonluku onnistuu eri tekstitiedostoilla
- normalisointi poistaa kohinaa rikkomatta merkitystä
- avainsanojen poiminta nostaa esiin odotetut termit tunnetuissa testisyötteissä
- vertailulogiikka erottaa vahvat, osittaiset ja puuttuvat osumat oikein
- suositusgeneraattori ei ehdota CV:n ulkopuolisia taitoja faktana
- luonnosteksti rakentuu ilman tyhjiä tai ristiriitaisia lauseita

Lisäksi kannattaa tehdä muutama integraatiotesti kokonaisilla esimerkkisyötteillä.

17. Tietosuoja ja turvallisuus

Koska CV ja työhakemuksiin liittyvä aineisto ovat arkaluonteisia, ratkaisu kannattaa suunnitella privacy-first-periaatteella:

- kaikki analyysi suoritetaan paikallisesti
- tiedostoja ei lähetä ulkoisiin API-palveluihin oletuksena
- raportteihin ei lisätä ylimääräisiä henkilötietoja automaattisesti

- mahdolliset lokit pidetaan minimaalisina
- poistettavat tai arkistoitavat raportit ovat käyttäjän hallinnassa

18. Riskit ja hallintakeinot

Keskeiset riskit ovat:

- ilmoituksen kieli on epämääräinen tai markkinointihenkinen, jolloin oleelliset vaatimukset hukkuvat
- CV on liian lyhyt tai yleistasoinen, jolloin osumia on vaikea perustella
- synonyymit ja eri kirjoitusasut heikentävät osumatarkkuutta
- luonnosteksti alkaa kuulostaa liian geneeriseltä

Hallintakeinot:

- yhdistetään sana- ja fraasitasoinen analyysi
- käytetään epätarkan osuman vertailua hallitusti
- liitetään jokaisen suosituksen yhteyteen perustelu
- pidetään generointi mallipohjaisena ja faktasidonnaisena

19. Hyvaksyntakriteerit

Ensimmäinen versio on valmis, kun:

- käyttäjä voi antaa tyopaikkailmoituksen ja CV:n kahdesta tiedostosta
- työkalu tuottaa listan keskeisistä avainsanoista
- työkalu tunnistaa, mitkä teemat löytyvät CV:sta
- työkalu antaa perustellut korostussuositukset
- työkalu tuottaa kelvollisen vahvuusluonnoksen
- analyysi voidaan tallentaa Markdown-raportiksi
- ydinlogiikalle on olemassa automatisoidut testit

20. Jatkokehitys

Kun MVP toimii, seuraavat kehitysaskeleet ovat luontevia:

- PDF- ja DOCX-sisaanluku
- useiden CV-versioiden vertailu samaan ilmoitukseen
- useiden ilmoitusten vertailu samaan CV:hen
- selain- tai desktop-käyttoliittymä
- muokattavat kirjoitustyylit vahvuusluonnokselle
- ATS-yhteensopivuuden tarkistus
- erillinen tila suomen- ja englanninkielisille hakemuksille

21. Suositus seuraavaksi askeleeksi

Paras seuraava askel on rakentaa ensin toimiva CLI-MVP. Se antaa nopeasti oikeaa palautetta analyysilogiikasta ja pitää teknisen riskin matalana. Kun ydinputki toimii hyvin tekstitiedostoilla, formaattituki ja käyttoliittymä voidaan lisätä turvallisesti päälle.